

Grader Output

C3M3_autograded

96.00 / 100.00 points earned

11 / 12 autograded cells passed

Graded Cells

Cell 4 (cell-6763e316191bcea7)

Passed | 14.00 / 14.00 points

[View feedback](#)**Cell 5 (cell-31a6a7aa076a4fb7)**

Passed | 12.00 / 12.00 points

[View feedback](#)**Cell 7 (cell-ca75109d7ba8a052)**

Passed | 20.00 / 20.00 points

[View feedback](#)**Cell 9 (cell-e2f56e9eb85424fa)**

Passed | 12.00 / 12.00 points

[View feedback](#)**Cell 11 (cell-2b8076e50ceb148c)**

Passed | 4.00 / 4.00 points

[View feedback](#)**Cell 12 (cell-6f9603be4594f0a2)**

Passed | 4.00 / 4.00 points

[View feedback](#)**Cell 13 (cell-0b0a2f50223582f1)**

Try again | 0.00 / 4.00 points

[View feedback](#)**Cell 15 (cell-3cfc822cb4e64686)**

Passed | 6.00 / 6.00 points

[View feedback](#)**Cell 16 (cell-819edd2f9910788c)**

Passed | 6.00 / 6.00 points

[View feedback](#)**Cell 17 (cell-600d5214b9fd5178)**

Passed | 6.00 / 6.00 points

[View feedback](#)**Cell 18 (cell-c78bc9cc7baf56d)**

Passed | 6.00 / 6.00 points

[View feedback](#)**Cell 19 (cell-759317f173c75458)**

Passed | 6.00 / 6.00 points

[View feedback](#)

C3M3 Autograded Assignment

Outline:

Here are the objectives of this assignment:

1. Apply nonparametric regression to actual data.
2. Visualize the differences between parametrics and nonparametric regressions.
3. Cement our understanding of Kernels, Bandwidths, and Splines.
4. Understand the different kinds of nonparametric regression.

Here are some general tips:

1. Read the questions carefully to understand what is being asked.
2. When you feel that your work is completed, feel free to hit the `Validate` button to see your results on the *visible* unit tests. If you have questions about unit testing, please refer to the "Module 0: Introduction" notebook provided as an optional resource for this course. In this assignment, there are hidden unit tests that check your code. You will not receive any feedback for failed hidden unit tests until the assignment is submitted. **Do not misinterpret the feedback from visible unit tests as all possible tests for a given question--write your code carefully!**
3. Before submitting, we recommend restarting the kernel and running all the cells in order that they appear to make sure that there are no additional bugs in your code.

```
In [1]: # Load Required Libraries
library(testthat)
library(tidyverse)
```

Problem 1: Comparing Parametric and Nonparametric Regression

The `exa` dataset is a simulated dataset following the function $f(x) = \sin^3(2\pi x^3)$. The data is loaded and plotted below. We will use this dataset to get you practicing some non-parametric regression techniques.

Data Source: Härdle, W. (1991). Smoothing Techniques with Implementation in S. New York:Springer.

```
In [2]: # Load in the Data
exa = read.csv("exa.csv")
head(exa)
# Plot the Data
g = ggplot(exa, aes(x=x, y=y)) +
  geom_point()
g
```

1. (a) Linear Regression First

From the graph, we can assume that a linear model isn't going to plot to this function very well. But assumptions can be wrong, so let's see how well they do for our model.

Begin by fitting a linear model with `y` as the response and `x` as the predictor. Save this model as `exa.lmod.1` and its R^2 as `exa.lmod.1.r2`. Because there is only one predictor, we can visually plot our model to see how well it fits the data. Plot your model on top of the original plot.

Now, let's add some parametric terms. Following the techniques displayed in the videos, add parametric terms of `x` of higher and higher degrees until you reach one that is no longer significant. Use the previous model with a significant d^{th} predictor coefficient. Save this model as `exa.lmod.d` and its R^2 value as `exa.lmod.d.r2`. Then plot this model on top of the original plot as well.

In [3]:

(Top)

```

exa.lmod.1 = lm(y ~ x, data=exa)
exa.lmod.1.r2 = summary(exa.lmod.1)$r.squared

exa.lmod.d = lm(y ~ x + I(x^2) + I(x^3) + I(x^4) + I(x^5) + I(x^6) + I(x^7), data=exa)
exa.lmod.d.r2 = summary(exa.lmod.d)$r.squared

# your code here
# exa.lmod.d = lm(y ~ x + I(x^2) + I(x^3) + I(x^4) + I(x^5) + I(x^6) + I(x^7), data=exa)
# summary(exa.lmod.d)

# plot(x=exa$x, y=exa$y, main="EXA with models overlaid")
# lines(exa.lmod.1, col="#CFB87C", lwd=2)
# lines(exa.lmod.d, col="#565A5C", lwd=2)

g +
  geom_smooth(method="lm", formula=eval(exa.lmod.1$call[[2]]), col="#CFB87C") +
  geom_smooth(method="lm", formula=eval(exa.lmod.d$call[[2]]), col="#565A5C")

```

In [4]:

cell-6763e316191bcea7

(Top)

```

# Test Cell
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted

```

Congratulations! All test cases in this cell passed.

In [5]:

cell-31a6a7aa076a4fb7

(Top)

```

# Test Cell
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted

```

Congratulations! All test cases in this cell passed.

1. (b) Visualize some Smooth Functions

Even if `exa.lmod.d` looks pretty good, we know that it's not the original function. Let's try some smoothing functions to see how those perform.

Use the `ksmooth()` function to plot some kernel estimators of the unknown function $Y = f(x)$. Explore different possibilities for kernel functions, including `normal` and `uniform` kernels. For each kernel, try different bandwidths including 0.01, 0.1 and 1. Plot your models and determine which combination provides the best results. Save your selected kernel as `best.kernel` and the best bandwidth as `best.bandwidth`.

In [6]:

(Top)

```

best.kernel = "box"
best.bandwidth = 0.1

# your code here
#?ksmooth
for (k in c('box', 'normal')) {
  for (b in c(0.01, 0.1, 1)) {
    kernel = ksmooth(exa$x, exa$y, kernel=k, bandwidth=b)
    #print(str(kernel))
    plot(exa$x, exa$y, pch=19, main=paste('kernel = ', k, 'bw = ', b))
    lines(kernel)
    print(paste(k, b, sum((exa$y - kernel$y)**2, na.rm=TRUE)))
  }
}

#?ksmooth

```

In [7]:

cell-ca75109d7ba8a052

(Top)

```

# Test Cell
if (!test_that("Checking Answer types", {expect_is(best.kernel, "character")
  expect_is(best.bandwidth, "numeric")})){
  print("Answers are incorrect types. Make sure best.bandwidth is numeric and best.kernel is a string.")
}
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted

```

Congratulations! All test cases in this cell passed.

In reality, manually finding this kernel value ourselves, especially if there are multiple predictors, can be challenging. There are some automated ways to do this, but none are perfect.

1. (c) Smoothing Splines

Use the `smooth.spline()` function to fit a non-parametric model to your data, with `y` as the response and `x` as the predictor. Save your model as `exa.smooth`. Plot this model onto the original scatterplot.

In [8]:

(Top)

```

exa.smooth = smooth.spline(exa$x, exa$y) #, spar = 0.5)

# your code here

plot(y ~ x, data = exa, main = "With Smoothing Splines")
lines(exa.smooth, col = "#CFB87C", lwd=3)

```

In [9]:

cell-e2f56e9eb85424fa

(Top)

```

# Test Cell
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted

```

Congratulations! All test cases in this cell passed.

1. (d) Goodness of Fit

How do we determine how well our model fits the data? We don't have an analytical method, so we need to use evaluation metrics.

Calculate the MSE for your three models. Store the values in `MSE.1`, `MSE.d` and `MSE.smooth` respectively. Which model performed the best?

In [10]: (Top)

```
# your code here
MSE.1 = mean((exa$y - predict(exa.lmod.1))**2, na.rm=TRUE)
MSE.d = mean((exa$y - predict(exa.lmod.d))**2, na.rm=TRUE)
MSE.smooth = mean((exa$y - exa.smooth$y)**2, na.rm=TRUE)
c(MSE.1, MSE.d, MSE.smooth)
```

In [11]: (Top)

cell-2b8076e50ceb148c

```
# Test Cell
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted
```

Congratulations! All test cases in this cell passed.

In [12]: (Top)

cell-6f9603be4594f0a2

```
# Test Cell
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted
```

Congratulations! All test cases in this cell passed.

In [13]: (Top)

cell-0b0a2f50223582f1

```
# Test Cell
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted
```

One or more test cases in this cell did not pass.

Instructor hints:

1. "Check MSE values"

Problem 2 - General Nonparametric Questions

For each of the following questions, save your answer in the corresponding variable. Answer each as a boolean TRUE/FALSE value.

1. Generally, the choice of Kernel is more important than the choice of bandwidth.
2. A Kernel is defined to be symmetric for all values of x and $\int K(x)dx = 1$.
3. Smoothing Splines are a balance between fitting the training data as accurately as possible and penalizing models that have a greater amount of curvature.
4. For Loess estimation, if $p = 0$, then the Loesse estimator is equivalent to Kernel estimation.
5. The `smooth.spline()` function is guaranteed to give you the best bandwidth value.

In [14]: (Top)

```
prob.2.1 = FALSE
prob.2.2 = TRUE
prob.2.3 = TRUE
prob.2.4 = TRUE
prob.2.5= FALSE
# your code here
```

In [15]: cell-3cfc822cb4e64686 (Top)

```
# Test Cell
if (!test_that("Checking answer types", expect_is(c(prob.2.1, prob.2.2, prob.2.3, p
rob.2.4, prob.2.5),
c("logical", "logical", "logica
1", "logical", "logical")))){
  print("Make sure your answers are all booleans!")
}
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted
```

Congratulations! All test cases in this cell passed.

In [16]: cell-819edd2f9910788c (Top)

```
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted
```

Congratulations! All test cases in this cell passed.

In [17]: cell-600d5214b9fd5178 (Top)

```
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted
```

Congratulations! All test cases in this cell passed.

In [18]: cell-c78bc9cc7baf56d (Top)

```
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted
```

Congratulations! All test cases in this cell passed.

In [19]: cell-759317f173c75458 (Top)

```
# This cell has hidden test cases that will run after submission.
Hidden Tests Redacted
```

Congratulations! All test cases in this cell passed.